

Guía Final: Usuarios Reales, bcrypt y JWT

Objetivo: completar la autenticación real del proyecto Full Stack. Al finalizar, la app debe permitir registrar usuarios, iniciar sesión con contraseña protegida, recibir un JWT y acceder a rutas protegidas.

Resultado que deben demostrar

- Una pantalla visual de Registro/Login en el frontend.
- Un usuario guardado en PostgreSQL en la tabla User.
- La contraseña guardada con hash, no como texto plano.
- Login funcional: email + password correctos devuelven un token JWT.
- Una ruta protegida que solo responde si se envía Authorization: Bearer token.
- La app debe explicar el flujo: React -> Express -> Prisma -> PostgreSQL -> JWT.

Importante: esta guía usa bcryptjs, una implementación en JavaScript del hashing tipo bcrypt. Es más fácil de instalar para clase. En la presentación pueden decir “bcrypt para proteger contraseñas”.

1. Instalar dependencias

Ejecutar dentro de la carpeta backend:

```
npm install bcryptjs jsonwebtoken
npm install -D @types/jsonwebtoken
```

Si ya tienen jsonwebtoken instalado, no pasa nada si verifican que esté en package.json.

2. Crear modelo User en Prisma

Abrir backend/prisma/schema.prisma y mantener el modelo Task. Agregar también el modelo User:

```
model Task {
  id      Int      @id @default(autoincrement())
  text    String
  completed Boolean @default(false)
  createdAt DateTime @default(now())
}

model User {
  id      Int      @id @default(autoincrement())
  name    String
  email   String   @unique
  password String
  createdAt DateTime @default(now())
}
```

El campo password guardará el hash, no la contraseña escrita por el usuario.

3. Ejecutar migración

Después de modificar schema.prisma, ejecutar dentro de backend:

```
npx prisma@6.19.0 migrate dev --name add_user_model
npx prisma@6.19.0 generate
```

Luego pueden abrir Prisma Studio para confirmar que existe la tabla User:

```
npx prisma@6.19.0 studio
```

4. Importar bcrypt y JWT en backend

En backend/src/index.ts, arriba junto a los otros imports:

```
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
```

bcrypt se usará para proteger contraseñas. JWT se usará para crear el token después del login.

5. Crear ruta POST /register

Esta ruta registra usuarios reales. Recibe name, email y password. Antes de guardar, convierte la contraseña en un hash.

```
// AUTH: Register creates a real user in PostgreSQL.
// AUTH: The password is hashed before saving it.
app.post("/register", async (req: any, res: any) => {
  const { name, email, password } = req.body || {};

  if (!name || !email || !password) {
    return res.status(400).json({
      message: "Name, email and password are required"
    });
  }

  const existingUser = await prisma.user.findUnique({
    where: { email: email }
  });

  if (existingUser) {
    return res.status(400).json({
      message: "User already exists"
    });
  }

  const hashedPassword = await bcrypt.hash(password, 10);

  const newUser = await prisma.user.create({
    data: {
      name: name,
      email: email,
      password: hashedPassword
    }
  });

  res.status(201).json({
    message: "User registered successfully",
    user: {
      id: newUser.id,
      name: newUser.name,
      email: newUser.email
    }
  });
});
```

6. Actualizar ruta POST /login con usuarios reales

Esta ruta ya no debe usar credenciales fijas. Ahora busca el usuario en PostgreSQL y compara la contraseña con bcrypt.

```
// AUTH: Login now checks real users from PostgreSQL.
// AUTH: bcrypt.compare checks the typed password against the saved hash.
app.post("/login", async (req: any, res: any) => {
  const { email, password } = req.body || {};

  if (!email || !password) {
    return res.status(400).json({
      message: "Email and password are required"
    });
  }

  const user = await prisma.user.findUnique({
    where: { email: email }
  });

  if (!user) {
```

```

        return res.status(401).json({
            message: "Invalid credentials"
        });
    }

    const passwordIsValid = await bcrypt.compare(password, user.password);

    if (!passwordIsValid) {
        return res.status(401).json({
            message: "Invalid credentials"
        });
    }

    const token = jwt.sign(
        { id: user.id, email: user.email },
        "secret_key",
        { expiresIn: "1h" }
    );

    res.json({
        message: "Login successful",
        token: token,
        user: {
            id: user.id,
            name: user.name,
            email: user.email
        }
    });
});

```

7. Probar registro y login en Thunder Client

A. Registrar usuario

POST http://localhost:3000/register
Body -> JSON

```

{
  "name": "Ana",
  "email": "ana@test.com",
  "password": "123456"
}

```

Luego abrir Prisma Studio y verificar que el usuario aparezca en User. La contraseña debe verse como una cadena larga, no como 123456.

B. Iniciar sesión

POST http://localhost:3000/login
Body -> JSON

```

{
  "email": "ana@test.com",
  "password": "123456"
}

```

La respuesta debe incluir un token:

```

{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": 1,
    "name": "Ana",
    "email": "ana@test.com"
  }
}

```

8. Mantener ruta protegida GET /profile

La ruta protegida sirve para demostrar que el token funciona. Debe revisar el header Authorization.

// AUTH: Protected route. Requires a valid JWT token.

```

app.get("/profile", (req: any, res: any) => {
  const authHeader = req.headers.authorization;

  if (!authHeader) {
    return res.status(401).json({
      message: "No token provided"
    });
  }

  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, "secret_key");

    res.json({
      message: "Protected profile data",
      user: decoded
    });
  } catch (error) {
    res.status(401).json({
      message: "Invalid token"
    });
  }
});

```

Probar con token

GET http://localhost:3000/profile
 Headers:
 Authorization: Bearer TU_TOKEN_AQUI

9. Crear pantalla visual en frontend

Deben crear una pantalla diferente para mostrar el login funcionando. Puede ser simple, pero debe ser visual.

- Formulario de registro: name, email, password.
- Formulario de login: email, password.
- Botón para iniciar sesión.
- Mensaje si el login fue exitoso.
- Mostrar el token de forma corta o indicar que se guardó.
- Botón o sección para probar ruta protegida /profile.

10. Guardar token en el frontend

Cuando el login funciona, el frontend debe guardar el token. Para esta práctica pueden usar localStorage:

```
localStorage.setItem("token", data.token);
```

Después, para llamar a una ruta protegida:

```

const token = localStorage.getItem("token");

const response = await fetch("http://localhost:3000/profile", {
  headers: {
    Authorization: `Bearer ${token}`
  }
});

```

Nota: En aplicaciones reales hay decisiones de seguridad más avanzadas. Para este proyecto, localStorage es suficiente para demostrar el flujo.

11. Lo mínimo que deben presentar

Parte	Debe demostrarse
Registro	Crear usuario desde frontend o Thunder Client.
bcrypt	La contraseña guardada en PostgreSQL debe verse hasheada.
Login	Email/password correctos devuelven JWT.
Token	El frontend guarda o muestra el token.
Ruta protegida	Sin token bloquea; con token permite acceso.
Explicación	Describir el flujo React -> Express -> Prisma -> PostgreSQL -> JWT.

12. Frase para explicar en presentación

“Antes el login usaba credenciales fijas. Ahora el sistema registra usuarios reales en PostgreSQL, protege sus contraseñas con bcrypt y genera un JWT cuando el login es correcto. Ese token se usa para acceder a rutas protegidas.”

13. Errores comunes

- Si req.body llega vacío: verificar `app.use(express.json())`.
- Si React no puede llamar al backend: verificar `app.use(cors())`.
- Si login dice Invalid credentials: revisar email exacto y password.
- Si no aparece la tabla User: correr migrate dev otra vez.
- Si /profile falla con token: revisar que el header sea Authorization: Bearer token.

Checkpoint final: La app debe tener usuarios reales, contraseñas protegidas, login funcional, JWT y una pantalla visual que demuestre el flujo.